Originality.ai

Document report

This report displays the results of the following scans: Plagiarism detection,

Untitled Scan

28 Oct 2026, 5:01 pm

https://app.originality.ai/share/zcfl6k024hbwauyi

Document report

Untitled Scan | Last scanned: 28 Oct 2026, 5:01 pm

PLAGIARISM RESULTS

0% match

0 matched websites

0 matched phrases

Our most accurate plagiarism checker uses advanced machine learning to detect complex forms of patchwork, unintentional & mosaic plagiarism, and more.

Most plagiarized sources:

PLAIN

Design and Implementation of an Optimized High-Performance Password Cracking Framework for Cybersecurity Analysis G. Sirisha, Y. Yamini, and A. Srinivas

Department of Computer Science and Engineering, KL University, India

Course Name: Cryptanalysis and Cyber Defence (24CSB3101), Academic Year. 2026—2027

Abstract - Password hashes are commonly used to check user credentials. However, if a user uses weak passwords and store them incorrectly they are at risk of a guess attack. We describe the design of an optimised password cracker tool to allow safe analysis of cyber security. This framework will allow dictionary, brute force and rule based password cracking attacks by using test hashes. We can compare the basic and optimised versions by looking at how long it took to complete. How many passwords were tried per second, CPU usage and memory usage. The framework will enable us to see how each password attacks compare to each other by comparing the trade-offs. Results will dependant on the test hashes, hashing algorithm, password space, and hardware. No claims will be made about numeric performances until the tests have been completed.

Keywords - Password security, Password hash auditing, Parallel Processing, Cyber security, Performance.

I. INTRODUCTION

Passwords are commonly used for authenticate users. However, the strength of a password depends on how long it is and how random it is stored and validated by the application. Password hashes can be cracked by trying to guess the passwords. This depends on the amount of passwords we are trying and what hashing algorithm we are using with what parameters [1].

Password cracking can also be used as a defence mechanism. By using this tool, we can analyse how many easy passwords there are to guess within the test set. We can see how expensive different password cracking techniques are to crack passwords. It will also make us realise how important it is to have strong password policies and correct password hashing. This tool should only be used with synthetic data.

The aim of this project is to design a framework to analyse different password cracking techniques. The framework will use dictionary, brute force and rule based techniques. We will look at how using parallelism and optimising password cracking will impact the runtime for a given test. The main question of this project is how will using optimisations impact the runtime and resources used during the test for a given amount of passwords?

Contributions of this project will be: (1) A modular password cracking framework using three password cracking techniques. (2) A reliable method of testing by looking at the runtime, passwords tried per second, CPU and Memory usage. (3) Comparing the basic and optimised techniques to show the areas where optimisations can be beneficial. No Conclusions e will be made about the performance of the system until it has been implemented.

II. BACKGROUND AND LITERATURE REVIEW

Password auditing usually involves testing passwords against a stored password validator. Dictionary attacks involve using a predefined list of likely passwords. Brute force attacks involve trying all possible passwords within a certain character set and password length. Rule based attacks transform a base word using predefined patterns like capitalisation, suffixes or number substitutions. These different strategies will have different password spaces and should be compared using the same test hashes.

The password storage design will have an impact on how expensive offline guess attacks are. OWASP recommend specific password hashing algorithms such as Argon2id, scrypt or PBKDF2 with suitable work factors [1]. Using only a fast hashing algorithm to test may not reflect how expensive it would be to audit a modern password store. The hashing algorithm and parameters used should be stated for all experiments.

Using parallel processing can help divide the independent password guesses between workers. However, this doesn't always mean there will be a proportional increase in speed. Creating processes, communication between processes, increased memory pressure, CPU limits, and hashing cost can all impact this. The increase in speed should be measured. This project will treat optimisation as an empirical problem. The runtime and resources will be recorded.

III. SYSTEM DESIGN

The framework will be designed with five logical components.:

(1) Input Validation-This will take in the test hash, declared algorithm, and parameters. And a limited test setup.

(2) Password Generation-This will generate passwords based on the chosen strategy.

(3) Password Validation — This will use the correct password hashing algorithm and compare the results with the test validator.

(4) Execution control — This will execute either basic or optimised version.

(5) Metrics — This will record the results.

The test should be done with synthetic accounts and test hashes. The framework should reject any unknown input format. There should be limits on how many passwords and how long the test will run for. The framework shouldn't allow credential dumping. The Logs include metadata about the experiment and the metrics but not the passwords found. This ensures the project is looking at using this as a defence mechanism.

The basic version will sequentially try passwords using a simple password generator and validator. The optimised version may have a password generator which doesn't repeat work. The passwords may also be split between a limited number of workers. Each worker will get a unique range of passwords. This should ensure deterministic coverage so performance issues are not caused by skipping some passwords.

IV. METHODOLOGY

A. Test Strategy- Dictionary: Passwords will be taken from a controlled wordlist. Brute force: Passwords will be generated within a predeclared character set and maximum password length. Rule based: Passwords will be transformed using a document set of rules. The passwords should be recorded with the source, rules, search space and stop condition.

B. Basic and Optimised Testing - Each strategy should be tested with the same synthetic hashes. Both versions should be tested with the same amount of passwords. The optimised version may have a configurable number of workers and improved password generator. The tests should be replaced with similar system conditions. The number of repeats and workers should be recorded. consistent as practical. The number of repetitions and worker count should be reported.

C. Performance Metrics - Let N be the number of passwords tested and T be time in seconds. Passwords per second can be calculated as R = N/T. The speed up can be calculated by dividing the basic time by the optimised time. S = Tb/To. The percentage reduction in runtime can be calculated by ((Tb − To)/Tb) × 100. CPU usage and Peak memory will be recorded. The same method should be used for both versions. If the test finishes early due to finding a match, then N should be the number of passwords actually tested.

D. Reproducibility: The OS, Processor, Memory, Python/Runtime version, Hashing Algorithm, Parameters, Passwords, Workers and Metrics tool should be recorded. If multiple tests are run, then the average should be taken and a measure of variation should be recorded. Don't compare tests with different hardware or passwords unless you explain why.

V. WEB-BASED IMPLEMENTATION

The project has been implemented as a web dashboard called "Password Security Analysis." The dashboard will have navigation to allow the user to move around the different sections. The sections are Overview, Experiment, Dataset, Performance, Resources, and Results. The Overview section has cards showing password information such as Dataset Size, Basic Time, Optimised Time, SpeedUp, Reduction, Optimised Passwords Per Second, Workers and Status.

The Speedup is calculated by dividing the basic time by the optimised time. The Reduction is calculated by taking the difference between the basic and optimised time and dividing by the basic time. Then Multiplying it by 100. The Passwords Per Second is calculated by dividing the number of passwords tested by the optimised time. The dashboard will show the calculated values.

The screenshots have labelled the analysis engine as a frontend simulation. This means the values shown here are not necessarily recorded from real password hash testing. If there is a backend benchmark engine, then the logs should be used to verify the results.

The Experiment Configuration section allows the user to enter the dataset name, number of dataset entries, hashing algorithm, Number of workers, basic and optimized execution times, CPU usage, memory usage, password tested and number of repeats. There is also a Dataset information section, Comparison Chart and resource indicators.